TECHNICAL DESIGN · 4 NOVEMBER 2026

specops

A composable agent orchestration kernel: deterministic gates scheduled in the cloud, with agent work on the engineer's own machine.

Platform decides CLI carries Local agent works Systems stay authoritative

specops Platform Design · Technical review board

THE PROBLEM

Agentic loops that never come up for air

One loop, one session

Intent, specification, plan, implementation, and verification run as one loop, in one session or across sub-agents with fresh context. Each step hands off on the agent's own say-so.

Deviations from the approved specification, errors, and context poisoning stay invisible until the end. In a large distributed system the damage is already in the change set.

A loop broken into gates

Every gate is a bounded check with a typed contract and recorded evidence. A deterministic check decides whether the work may advance, so a problem surfaces at the gate where it occurs.

Gates are hosted in the platform, so the record of what happened never lives in one person's terminal, and people on different machines continue the same job.

Spec-driven development is the first worked example; the same failure shape appears in any chained agent loop.

THE BOUNDARY

The platform decides whether work may advance. The CLI carries it. The local agent does it. Managed systems stay authoritative for their facts.

Humans stay in the loop wherever a pack declares a human gate.

Agent sessions run on the user’s machine, inside the managed sandbox.

The platform never hosts a model or an agent session, and only the platform calls GAIP.

Two standing constraints, architectural rather than pilot limitations

ARCHITECTURE

Three trust zones, one contract

01 · Developer machine

Does the agent work

- specops CLI: sign-in, leases, worktree, adapters, filtering, submit
- Local agent tool with its own model access
- Isolated Git worktree and repository index

02 · Platform

Coordinates, authorizes, records

- LangGraph kernel, context resolver, gate executor
- Append-only ledger that is also the checkpointer
- Signed evidence, event bus, entitlement-filtered search

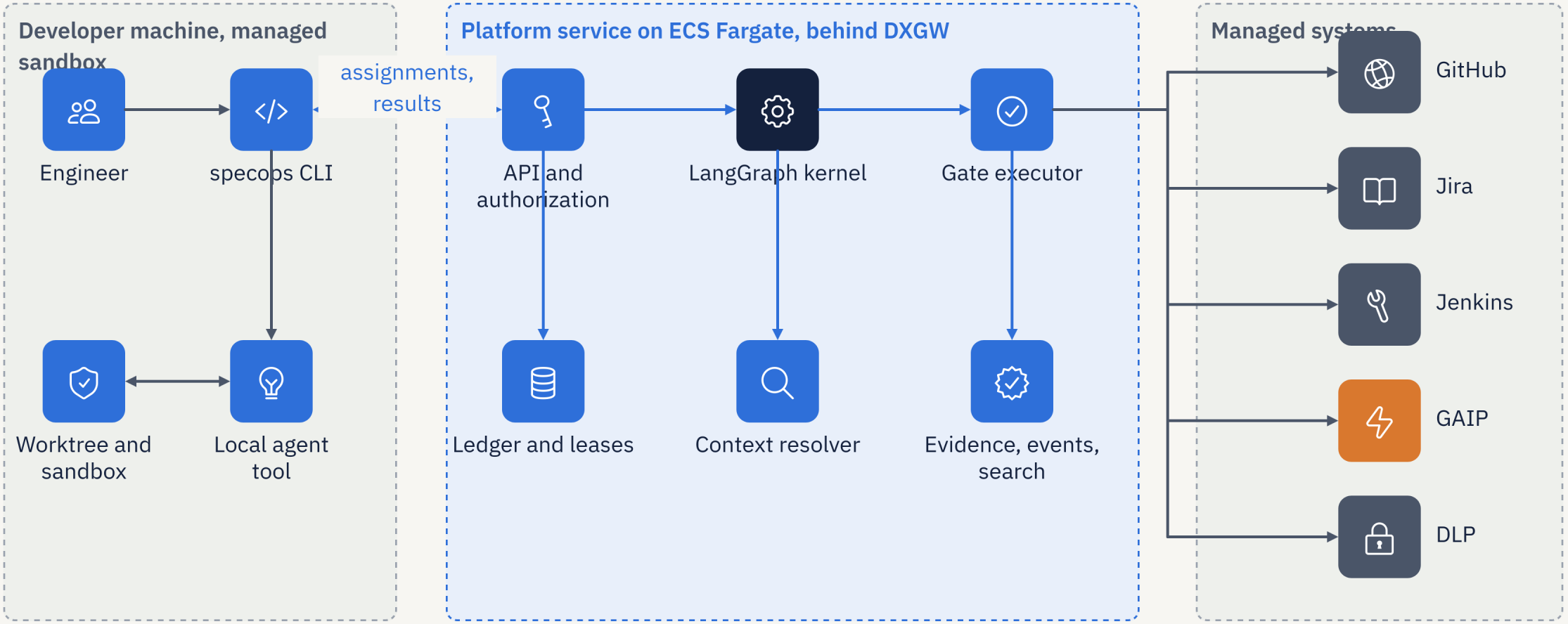
03 · Managed systems

Own their facts

- GAIP: Jev classifier, OFAC screening, model calls
- GitHub, Jira, Jenkins, read-only to the platform
- DLP service for PII and prompt-injection filtering

Every gate is a node in the platform. Its work may be done by a local session, a managed system, or a person, but each gate receives a **GateAssignment** and returns a **GateResult**. Delivery actions that change GitHub, Jira, or Jenkins run from the CLI under the user's identity, so a compromised platform cannot push source or deploy.

Architecture: three trust zones



Blue arrows carry typed contracts; grey arrows are read-only machine credentials. Delivery actions on GitHub, Jira, and Jenkins run from the CLI under the engineer's own identity.

KERNEL

Patterned on an operating-system kernel

Task scheduler

Compiles a Gate Pack to a LangGraph StateGraph, computes the ready set, interrupts and resumes

Message router

Picks contracts, schemas, and templates by work type and message type; delivers each result and event

Model router

Resolves a profile such as fast-v2 to a GAIP model for a platform gate or an entitled local model for an agent gate

Decision API

A classifier (Jev, for example) labels the work and decides whether each skippable gate is needed

Context resolver

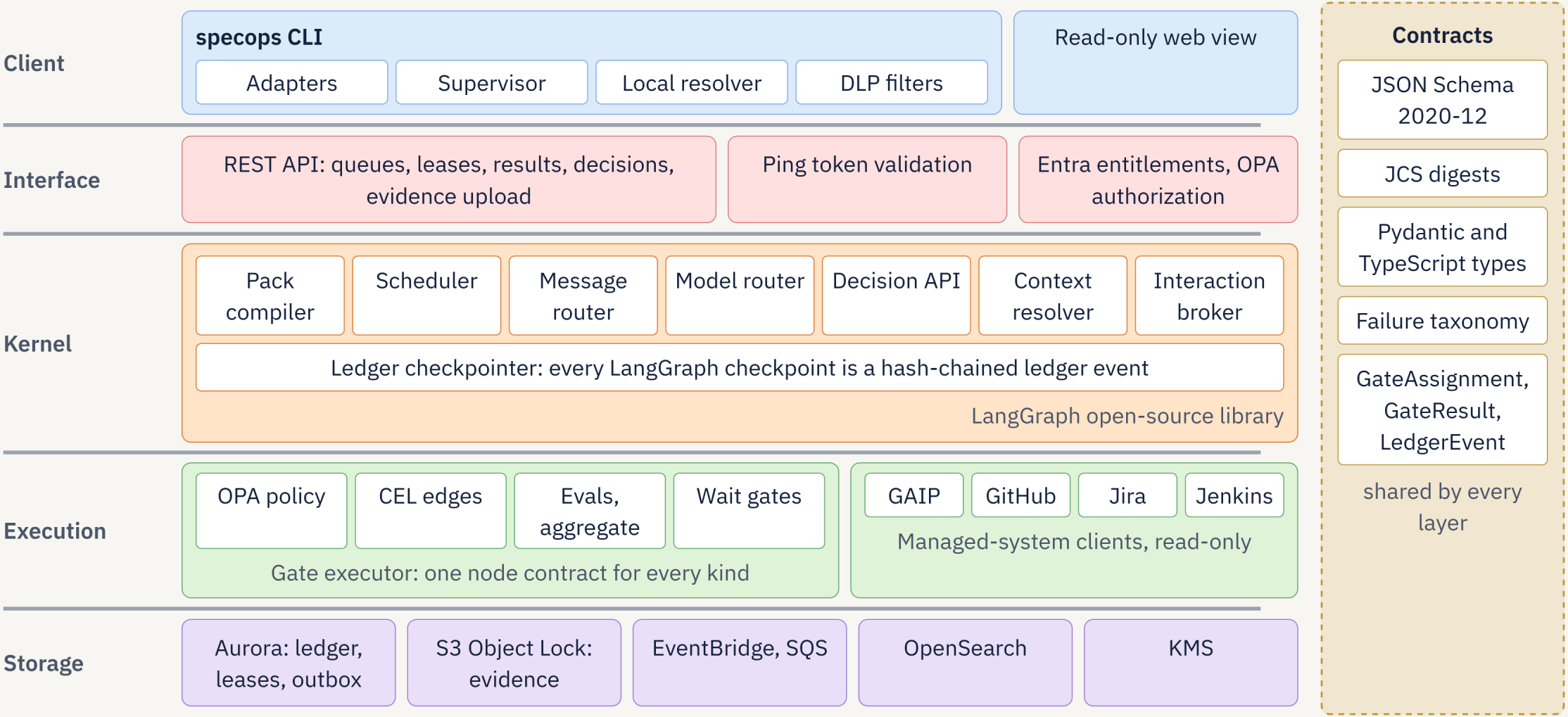
Builds a typed, immutable context manifest per attempt: references, rules, allowed paths, budgets

Interaction broker

Turns an agent's structured question or an approval need into a durable, authorized decision

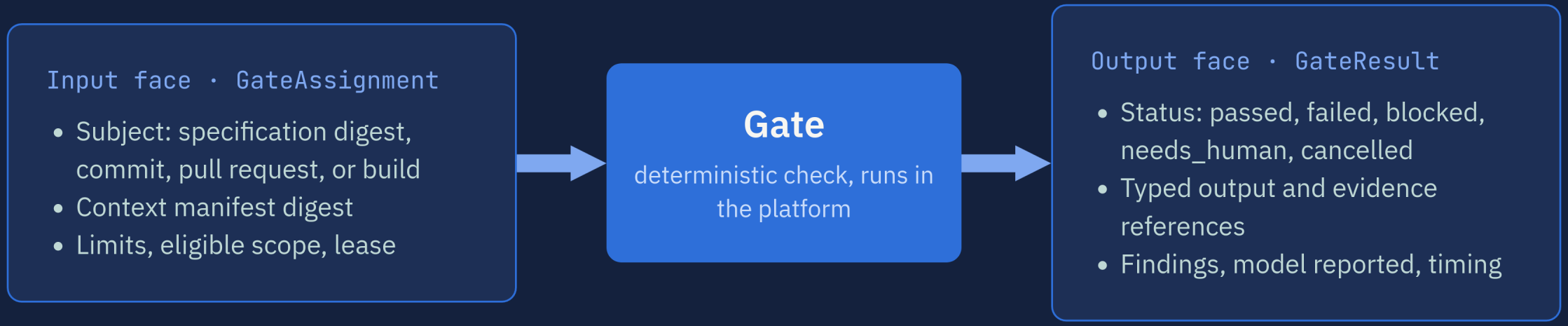
The ledger checkpointer underneath all of these writes every LangGraph checkpoint as a hash-chained ledger event. Gate Packs, contracts, and the domain model expose none of LangGraph's types.

Component structure: five layers and one cross-cutting contract



GATES

A gate is a node with two faces



Edges connect one node’s output face to the next node’s input face, and a CEL condition on an edge reads only the face it leaves from. Both faces are schemas, so the compiler checks every edge at validation time and the ledger records exactly what entered and left each gate instance.

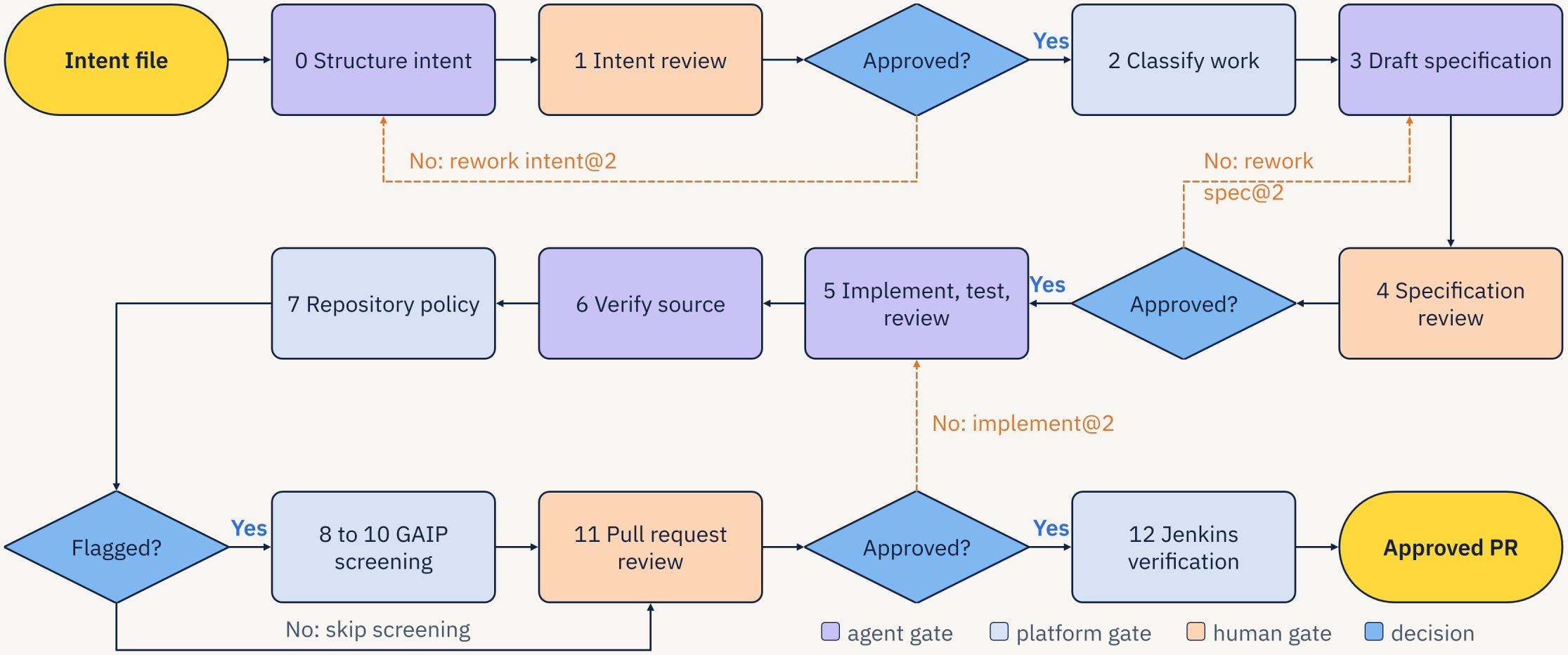
GATES

A closed set of gate kinds

Kind	Work done by	Purpose
agent	Local agent	One fresh, bounded session on the developer's machine, returning a schema-valid result
tool	CLI or platform	An approved deterministic command or service call, including wait-for-merge and wait-for-deployment
decision_classifier	Platform via GAIP	Labels the work so CEL edges can route around gates that do not apply
opa_policy	Platform	Evaluates a Rego bundle against accepted results and subject facts
ofac_screening	Platform via GAIP	Screens names and entities; the platform keeps no copy of sanctions lists
eval	Platform	A built-in deterministic operation: schema conformance, evidence freshness, test report parsing
human	Person	A typed approval or rejection bound to the exact subject digest reviewed
aggregate	Platform	Combines accepted predecessor results by declared logic

Teams compose approved kinds into Gate Packs; registering a new kind is an administrative platform change with its own lifecycle. No workflow language, no plugin marketplace, no arbitrary server-side code.

Business Case 1: intent to a confirmed build, with three rework paths



CODE COMPLETE TO MERGE

Every step after code complete is its own gate

Gate	Kind	What happens	Fix step
Tests	agent	Runs the suites, returns a structured report; the platform's eval gate parses it independently	Fix session with failures as input, up to max_attempts
Sabotage	agent	Mutation testing: rips out the change and expects the suite to fail; surviving mutants are gaps	Fix session strengthens tests
Code review	agent + Decision API	Fresh review session; findings labeled low, medium, high, critical	High triggers a fix session; critical goes to a human
Push and PR	tool, CLI	The CLI, not the agent, pushes and opens the PR by convention under the user's identity	A failed push is blocked, not retried
PR review	human	A reviewer who is not the author approves the exact head commit	Rejection follows the rework path
Merge	human in pilot	specops work merge after the platform confirms every required gate passed	Conflict is blocked with rework to implementation

A fix loop is the gate's retry policy: each fix is a new attempt within the same gate instance. The CLI never decides that tests passed; it carries the report and the platform reads it. Human checkpoints inside an attempt (preview, UAT, feedback) are a declared handoff, not an error.

TDD sections 7.3 to 7.6

DATA FLOW

One agent gate attempt, in thirteen exchanges

Platform assigns

1. Kernel publishes a GateAssignment and calls interrupt(); the thread is now stored rows
2. specops work claim: Entra entitlements, OPA, conditional lease insert
3. Platform returns the assignment and the resolved ContextManifest

CLI carries, agent works

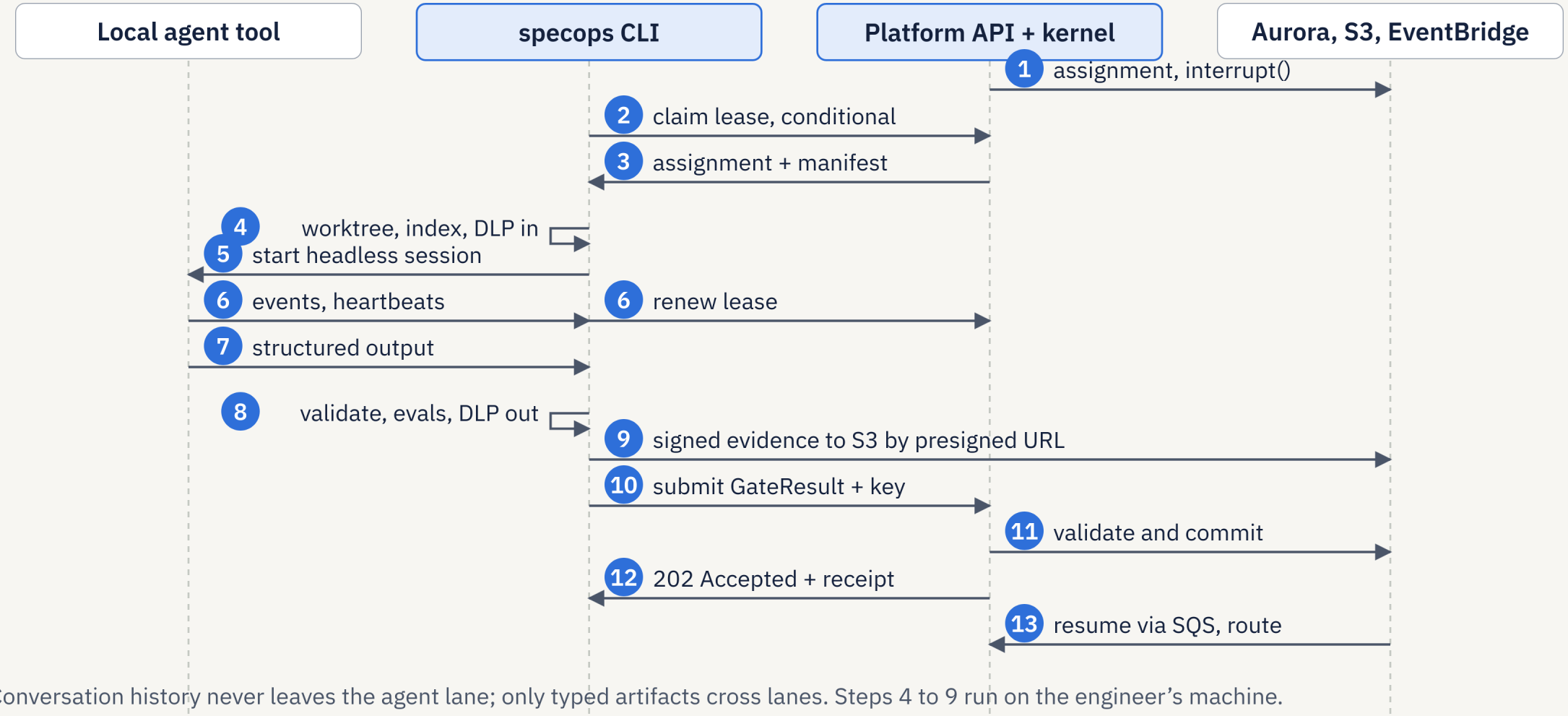
1. Isolated worktree at the base commit, repository index, inbound DLP filtering
2. Runtime adapter starts one headless session in the managed binary
3. Normalized events, heartbeats from activity, lease renewal on its own clock
4. Session ends with structured output; sub-agent prose is advisory
5. Schema validation, local evals, Git-derived facts, outbound DLP, JCS digests
6. Evidence bodies to S3 by presigned URL

Platform accepts

1. Submit GateResult with lease, expected ledger position, idempotency key
2. One transaction: validate, authorize, check lease and subject, append, outbox, commit
3. 202 Accepted with a receipt; a repeat returns the same receipt
4. Resume via SQS; the worker loads the checkpoint, evaluates the CEL edge, schedules the next gate

What never crosses a lane: the user’s OAuth token, the model’s conversation history, the sandbox’s credentials, any platform credential. The next attempt starts from the ledger, not from this session.

Data flow: one agent gate attempt



Conversation history never leaves the agent lane; only typed artifacts cross lanes. Steps 4 to 9 run on the engineer's machine.

FAILURE HANDLING

Stopped clients and failed checks are normal conditions

Condition	Meaning	Platform behavior	Spends attempt
Transient error	Network or provider failure inside an attempt	LangGraph RetryPolicy repeats the call	No
Failed attempt	The work was wrong	Same gate again with the failure details, up to max_attempts (default 2); exhaustion routes to a human gate	Yes
Rework	A later gate says an earlier output must change	New gate instance (gate@2) with a fresh budget; changed downstream gates re-run; max_rework_cycles (default 3)	New instance
Blocked	A dependency is unavailable: GAIP, Jira, Jenkins, DLP	Hold with the reason; resume when it returns; human gate after PT2H; authorized override	No
Needs human	The agent asks a structured question	DecisionRequest persisted, node interrupted, fresh assignment carries the decision	No
Expired lease	The CLI stopped renewing	Attempt returns to its queue unspent; a late result is rejected and kept as evidence	No
Repeated submission	Same idempotency key	The earlier receipt is returned; the ledger holds the result once	No

Unknown never means passed. A fact that cannot be confirmed fails the gate; a source system that cannot be reached blocks it.

RECORD

One ledger, signed evidence, committed events

Ledger

Append-only, hash-chained events in Aurora PostgreSQL, one per job. The database role has no UPDATE or DELETE rights. It is also LangGraph's checkpointer, so there is no second store to drift.

Current state is a projection rebuilt from events; corrections are new events that refer to earlier ones.

Evidence

in-toto statements in DSSE envelopes, signed with KMS, each naming the subject it covers. Bodies in S3 with Object Lock in compliance mode, retained 2 years.

A Jenkins result for commit A never satisfies a gate for commit B. When a subject changes, evidence is marked stale, never deleted.

Events and search

A transactional outbox in the same Aurora transaction feeds EventBridge; one SQS queue per consumer with a dead-letter queue: resume, notify, index, archive, live updates.

OpenSearch holds authorized summaries; entitlements are applied server-side to hits, counts, and suggestions, and every detail read re-checks Aurora.

An agent reporting tests_passed: true is a claim. The test gate runs the tests and records an evidence reference; the ledger records the kernel's verified evidence, not the agent's claim.

TDD section 13 · ADR-004, ADR-009

IDENTITY AND SECURITY

Knowing an identifier grants nothing

Who called

CLI: OAuth 2.0 Authorization Code with PKCE through Ping, loopback redirect, no client secret. Web: OIDC. Users keyed by Ping's stable subject, never an email.

What they may do now

Entra ID supplies All Access groups on every request; the platform keeps no membership copy. OPA evaluates caller, action, scope, current gate, lease holder, relationship, and separation of duty. Not-found is identical for missing and unauthorized.

Who owns the attempt

A lease: one user, one job's current gate attempt, one expiry, taken as a conditional insert in Aurora. Queues only signal that work exists; the lease decides ownership. A lease never widens permissions.

Credential boundaries

IAM task roles inside AWS. Read-only machine credentials to GitHub, Jira, Jenkins. The CLI acts on delivery systems as the user. The local agent receives neither the user's token nor any platform credential.

Two filtering boundaries in the CLI: before context reaches the local agent tool, and before results leave the machine. PII, credentials, and embedded instructions are removed or flagged; a flagged item fails the attempt with a recorded reason.

DEPLOYMENT

AWS managed services, open-source libraries

Request path

CLI or browser → Ping → DevExchange Gateway → internal ALB with WAF → platform API on ECS Fargate (us-east-1, us-west-2 recovery) → LangGraph kernel, OPA, Aurora through RDS Proxy, S3, platform gates.

Event path

Aurora outbox → EventBridge custom bus → SQS queues (resume, notify, index, archive, live) → kernel worker, notifications, OpenSearch indexer, archive export, server-sent events. EventBridge Scheduler runs lease expiry, blocked-gate deadlines, and the OFAC list refresh.

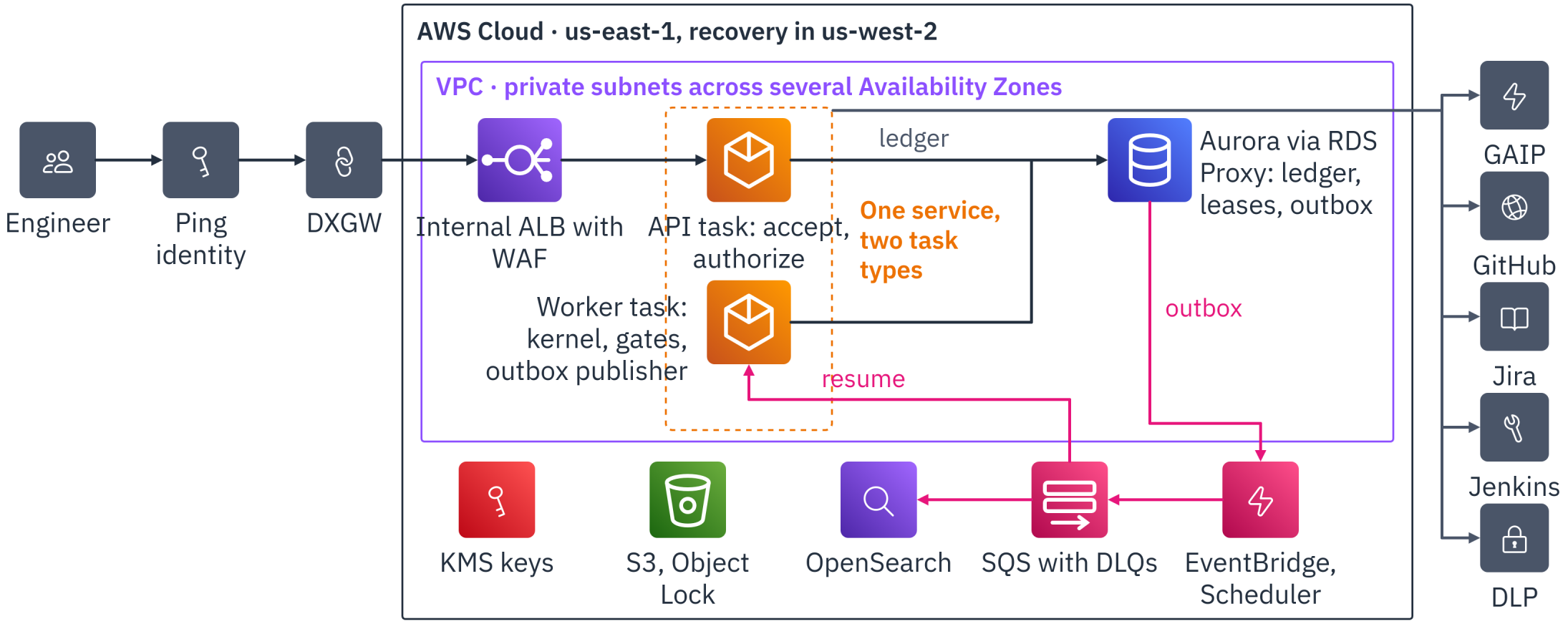
Infrastructure

AWS CDK in TypeScript; KMS for encryption and evidence signing; CloudWatch and X-Ray; private networking, egress allowlist for github.com, no CloudFront.

Need	Choice	License
Orchestration kernel	LangGraph Python library	MIT
Policy and authorization	Open Policy Agent, Rego	Apache 2.0
Edge conditions	CEL	Apache 2.0
Contracts	JSON Schema 2020-12, JCS digests, Pydantic and TypeScript types	open
Evidence format	in-toto statements, DSSE envelopes	Apache 2.0
Repository index	Tree-sitter, language-server metadata	MIT
Local agent access	Claude Agent SDK and Codex SDK adapters on managed binaries, subprocess fallback	vendor

TDD section 14 · ADR-012 lists what was deliberately not chosen

AWS topology



API and worker never call each other: an accepted result commits a ledger event and an outbox row, EventBridge carries it, and the worker resumes from SQS. Grey bus: read-only credentials to managed systems.

DELIVERY

Six stages, each with a proof

01

Contracts and graph compilation

Two packs compose the same gates into different valid graphs; an undeclared backward edge is rejected

02

Local CLI and local record

A prose-wrapped agent result cannot advance; a healthy stream is not cut by the activity timeout

03

Shared coordination

Two users on different machines continue one job; the database role cannot update or delete ledger events

04

Platform gates

Jev, one GAIP eval, one OPA gate, one OFAC gate; a GAIP outage blocks instead of failing

05

Delivery evidence

Jenkins success for a different commit cannot pass; a new commit makes earlier evidence stale, not missing

06

Multi-user pilot

Selected teams run the 13-gate pack; queue wait, lease conflicts, rework cycles, and skipped gates are measured from the ledger

The pilot stops at an approved pull request with a confirmed Jenkins result. Production release support needs a later decision based on the pilot’s security, recovery, and operating evidence.

TDD section 15 · 15 acceptance scenarios, 11 test areas

WHAT THE REVIEW BOARD DECIDES

Open decisions and next steps

Blocking the pilot

- Trial teams, repositories, Jira projects, Jenkins jobs
- Ping client registrations and the approved CLI login flow
- Lease length, renewal, human due dates, escalation ownership
- GAIP request API, authentication, IP ranges; the DLP service
- DXGW support for server-sent events and token validation
- CLI distribution, signing, and minimum-version process

Next steps

- Confirm the three Proposed records: context resolver, agent outcomes, the intent gate
- Assign owners and due dates to all 20 open decisions
- Author the schema bundle and the 13-gate Gate Pack with its CEL conditions and rework paths
- Stand up the pilot environment with CDK; run the recovery and authorization suites before onboarding a team

Approval requested: specops as the shared orchestration kernel and ledger for agent-assisted delivery, with agent work on the user’s machine, platform gates that call managed systems, and human gates for the decisions people own.